

EE4410 Cyber Security of Power Grids

Assignment 2: Cryptography Applications

Name: Zubair Al-Zubi

Student Number: 5342007

Date of Submission: May 28, 2025

Introduction

Objectives:

- Understand the application of symmetric and asymmetric cryptography.
- Test and demonstrate symmetric encryption and decryption using AES.
- Test and demonstrate asymmetric encryption and decryption using RSA.
- Test and demonstrate different digital signatures algorithms (i.e., RSASSA-PSS and ECDSA)

In this laboratory you will be communicating with another student using an unsecure communication medium. To simplify the exchange of messages between yourselves, JSONbin.io website and its public API will be used. Keep in mind that this is for real an unsecure communication channel given that ANY message published on the website can be read by ANYONE on the public internet!

Steps to use the JSONbin.io API:

- Create an account and sign in to JSONbin.io.
- From your JSONbin.io dashboard, under the "API KEYS" tab, you can obtain your personal X-Master-Key to be used to access the API. This key is unique to each JSONbin.io account and should be kept private!
- From your JSONbin.io dashboard, under the "BINS" tab, you can create a new PUBLIC bin. You can insert the following JSON to successfully initialize it: `{"Lab": "Crypto"}`. Note: remember to set the bin as public.
- Save your BIN ID. This will be your "unsecure upload communication channel". Your published message will be saved in this BIN.
- Ask your friend to provide their BIN ID. This will be the "unsecure download communication channel" to be used for receiving the messages from your friend.

Utility functions:

In the "Utility functions" chapter, you can find all the functions to be used to upload/download messages to JSONbin, encrypt/decrypt messages using AES/RSA, generate RSA/EC keys, and generate/validate digital signatures using RSASSA-PSS and ECDSA. Also, examples on how to use these functions are provided. The code in the "Utility functions" chapter is not meant to be modified; it is there to help you complete the tasks in the "Assignment" chapter.

JSONbin step-by-step tutorial

The screenshot shows the JSONbin.io dashboard. The sidebar on the left has a navigation menu with the following items: DASHBOARD, BINS (1), COLLECTIONS, SCHEMA DOCS, API KEYS, MY ACCOUNT, and API REFERENCE. The main content area is titled 'Bins' and contains a table of bins. The table has columns for Bin ID, Name, and Created. There are five bins listed, all with the name 'Untitled'. A 'Create a Bin' modal is open, showing a form to create a new bin. The modal has a 'Create A Bin' button (2), a 'Save Bin' button (5), and a 'Public' checkbox (3). The form contains a text area with the JSON '{"Lab": "Crypto"}' (4). There is also an 'Important' message in the modal: 'Setting the Bin permission to Public will allow anyone to READ & UPDATE the Bin. Anyone Updating the Public Bin without X-Master-Key will not be able to Disable Version Control while Updating the Bin.'

Bin ID	Name	Created
67dfce8f8561e97a50f12f47	Untitled	09:04 23-03-2025 (UTC)
67dd83378a456b79667a17e7	Untitled	15:18 21-03-2025 (UTC)
67dd82d08561e97a50f046c9	Untitled	15:16 21-03-2025 (UTC)
67dd52d48960c979a575c1dc	Untitled	11:51 21-03-2025 (UTC)
67dd51b88960c979a575c155	Untitled	11:47 21-03-2025 (UTC)

✓ Define and import environment variables

```
1 from google.colab import userdata
2
3 JSONbin_X_Master_Key = userdata.get('JSONbin_X_Master_Key')    # Your personal API key to needed to use
4
5 JSONbin_Bin_ID_A = '6836102f8561e97a501c4973'                # Your own BIN. Here is where your messages will b
6 JSONbin_Bin_ID_B = '68373cd58561e97a501caf9c'                # Your friend BIN. Here is from where your friend i
```

✓ Import required modules

```
1 !pip install pycryptodome
2
3 import base64
4 import requests
5 import json
6 import binascii
7
8 from Crypto.Cipher import AES, PKCS1_OAEP
9 from Crypto.Hash import SHA256
10 from Crypto.PublicKey import RSA, ECC
11 from Crypto.Signature import DSS, pss
12 from Crypto import Random
```

🔗 Requirement already satisfied: pycryptodome in /usr/local/lib/python3.11/dist-packages (3.23.0)

✓ Utility functions

✓ JSONbin

Available functions:

- `send_message_to_jsonbin(bin_id, message, secret_key) → response`
- `receive_message_from_jsonbin(bin_id, secret_key) → response`

```
1 # Introduction
2 # This code demonstrates how to send and receive messages through JSONbin.
3 # JSONbin is a service that allows you to store and retrieve JSON data easily.
4
5 # JSONbin Utility functions
6 def send_message_to_jsonbin(bin_id, message, secret_key):
7     """
8     Sends a message to a JSONbin.
9
10    Args:
11        bin_id (str): The ID of the JSONbin.
12        message (dict): The message to send.
13        secret_key (str): Personal JSONbin API Master-KEY.
14
15    Returns:
16        dict: The response from JSONbin.
17    """
18    url = f"https://api.jsonbin.io/v3/b/{bin_id}"
19    headers = {
20        "Content-Type": "application/json",
21        "X-Master-Key": secret_key,
22    }
23    response = requests.put(url, headers=headers, json=message)
24    return response.json()
25
26
27 def receive_message_from_jsonbin(bin_id, secret_key):
28     """
29     Receives a message from a JSONbin.
30
31    Args:
32        bin_id (str): The ID of the JSONbin.
```

```

33     secret_key (str): Personal JSONbin API Master-KEY.
34
35     Returns:
36         dict: The message received from JSONbin.
37     """
38     url = f"https://api.jsonbin.io/v3/b/{bin_id}/latest"
39
40     headers = {
41         "X-Master-Key": secret_key,
42     }
43     response = requests.get(url, headers=headers)
44     if response.status_code == 200:
45         return response.json()["record"]
46     else:
47         return None

```

[▾ Example Usage](#)

```

1 # Replace with your JSONbin ID and secret key
2 bin_id = JSONbin_Bin_ID_A          # JSONbin where to send/receive the message
3 master_key = JSONbin_X_Master_Key  # JSONbin X-Master-Key
4
5 # Send a message
6 message = {"message": "Its lights out and away we go", "signature": "79672f07e38abbb8f6369453e016aacd"}
7 send_response = send_message_to_jsonbin(bin_id, message, master_key)
8 print(f"Send response: {send_response}")
9
10 # Receive the latest message
11 received_message = receive_message_from_jsonbin(bin_id, master_key)
12 print(f"Received message: {received_message}")
13

```

 Send response: {'record': {'message': 'Its lights out and away we go', 'signature': '79672f07e38abbb8f6369453e016aacd'}},
 Received message: {'message': 'Its lights out and away we go', 'signature': '79672f07e38abbb8f6369453e016aacd'}

[▾ Advanced Encryption Standard \(AES\)](#)

Available functions:

- `encrypt(message, passphrase, key_length, mode) → (nonce/iv +) encrypted_message (+ tag)`
- `decrypt(ciphertext, passphrase, key_length, mode) → message`

Supported modes of operation:

- Electronic Codebook Mode (ECB)
- Cipher Block Chaining Mode (CBC)
- Cipher Feedback Mode (CFB)
- Output Feedback Mode (OFB)
- Counter Mode (CTR)
- Counter with CBC-MAC mode (CCM)
- Galois/Counter Mode (GCM)
- Offset Codebook Mode (OCB)

Supported key lengths:

- 128
- 192
- 256

```

1 # AES docs: https://pycryptodome.readthedocs.io/en/latest/src/cipher/aes.html
2
3 import base64
4 from Crypto.Cipher import AES
5 from Crypto.Util.Padding import pad, unpad
6 from Crypto.Hash import SHA256
7 from Crypto.Random import get_random_bytes
8
9 def encrypt(message, passphrase, key_length=256, mode=AES.MODE_CBC):
10     """Encrypts a message using AES."""
11     if key_length not in [128, 192, 256]:

```

```

12     raise ValueError("Invalid key length. Must be 128, 192, or 256.")
13
14 # Generate AES key from passphrase
15 key = SHA256.new(passphrase.encode()).digest()[:key_length // 8]
16
17 if mode in [AES.MODE_CBC, AES.MODE_CFB, AES.MODE_OFB]:
18     iv = get_random_bytes(AES.block_size) # 16-byte IV
19     cipher = AES.new(key, mode, iv)
20     if mode in [AES.MODE_CFB, AES.MODE_OFB]:
21         encrypted_message = cipher.encrypt(message.encode())
22     else:
23         encrypted_message = cipher.encrypt(pad(message.encode(), AES.block_size))
24     return base64.b64encode(iv + encrypted_message).decode('utf-8')
25
26 elif mode in [AES.MODE_CCM, AES.MODE_GCM, AES.MODE_OCB, AES.MODE_CTR]:
27     nonce = get_random_bytes(12) # 12-byte nonce
28     cipher = AES.new(key, mode, nonce=nonce)
29     if mode in [AES.MODE_CCM, AES.MODE_GCM, AES.MODE_OCB]:
30         encrypted_message, tag = cipher.encrypt_and_digest(message.encode())
31         return base64.b64encode(nonce + encrypted_message + tag).decode('utf-8')
32     else:
33         encrypted_message = cipher.encrypt(message.encode())
34         return base64.b64encode(nonce + encrypted_message).decode('utf-8')
35
36 else:
37     cipher = AES.new(key, mode)
38     if mode in [AES.MODE_ECB]:
39         encrypted_message = cipher.encrypt(pad(message.encode(), AES.block_size))
40     else:
41         encrypted_message = cipher.encrypt(message.encode())
42     return base64.b64encode(encrypted_message).decode('utf-8')
43
44 def decrypt(encrypted_message, passphrase, key_length=256, mode=AES.MODE_CBC):
45     """Decrypts a message using AES."""
46     if key_length not in [128, 192, 256]:
47         raise ValueError("Invalid key length. Must be 128, 192, or 256.")
48
49     # Generate AES key from passphrase
50     key = SHA256.new(passphrase.encode()).digest()[:key_length // 8]
51
52     encrypted_message = base64.b64decode(encrypted_message)
53
54     if mode in [AES.MODE_CBC, AES.MODE_CFB, AES.MODE_OFB]:
55         iv = encrypted_message[:AES.block_size]
56         cipher = AES.new(key, mode, iv)
57         if mode in [AES.MODE_CFB, AES.MODE_OFB]:
58             decrypted_message = cipher.decrypt(encrypted_message[AES.block_size:])
59         else:
60             decrypted_message = unpad(cipher.decrypt(encrypted_message[AES.block_size:]), AES.block_size)
61
62     elif mode in [AES.MODE_CCM, AES.MODE_GCM, AES.MODE_OCB, AES.MODE_CTR]:
63         nonce = encrypted_message[:12] # 12-byte nonce
64         cipher = AES.new(key, mode, nonce=nonce)
65         if mode in [AES.MODE_CCM, AES.MODE_GCM, AES.MODE_OCB]:
66             tag = encrypted_message[-16:]
67             encrypted_message = encrypted_message[12:-16]
68             decrypted_message = cipher.decrypt_and_verify(encrypted_message, tag)
69         else:
70             decrypted_message = cipher.decrypt(encrypted_message[12:])
71
72     else:
73         cipher = AES.new(key, mode)
74         if mode in [AES.MODE_ECB]:
75             decrypted_message = unpad(cipher.decrypt(encrypted_message), AES.block_size)
76         else:
77             decrypted_message = cipher.decrypt(encrypted_message)
78
79     return decrypted_message.decode()
80

```

Example Usage

```

1 # Dictionary mapping mode indices to AES encryption modes
2 available_modes = {
3     1: AES.MODE_ECB, # Electronic Codebook Mode
4     2: AES.MODE_CBC, # Cipher Block Chaining Mode
5     3: AES.MODE_CFB, # Cipher Feedback Mode
6     5: AES.MODE_OFB, # Output Feedback Mode
7     6: AES.MODE_CTR, # Counter Mode
8     8: AES.MODE_CCM, # Counter with CBC-MAC Mode (Provides authentication)
9     11: AES.MODE_GCM, # Galois/Counter Mode (Authenticated encryption)
10    12: AES.MODE_OCB # Offset Codebook Mode (Authenticated encryption)
11 }
12
13 available_modes_to_string = {
14     1: "AES.MODE_ECB", # Electronic Codebook Mode
15     2: "AES.MODE_CBC", # Cipher Block Chaining Mode
16     3: "AES.MODE_CFB", # Cipher Feedback Mode
17     5: "AES.MODE_OFB", # Output Feedback Mode
18     6: "AES.MODE_CTR", # Counter Mode
19     8: "AES.MODE_CCM", # Counter with CBC-MAC Mode (Provides authentication)
20     11: "AES.MODE_GCM", # Galois/Counter Mode (Authenticated encryption)
21     12: "AES.MODE_OCB" # Offset Codebook Mode (Authenticated encryption)
22 }
23
24 # Loop through each available encryption mode
25 for m in available_modes:
26     print(f"Mode {available_modes_to_string[m]}")
27     passphrase = "Max Verstappen" # Secret key used for encryption
28     message = "I'm moving up and down, side to side like a roller coaster" # Message to be encrypted
29     key_length = 256 # AES key length in bits (valid options: 128, 192, 256)
30
31     mode_idx = m # Current mode index
32     mode = available_modes_to_string[m] # Retrieve AES mode from dictionary
33
34     # Encrypt the message using the specified AES mode
35     encrypted_message = encrypt(message, passphrase, key_length, mode)
36     print("Encrypted message:", encrypted_message)
37
38     # Decrypt the message to verify correctness
39     decrypted_message = decrypt(encrypted_message, passphrase, key_length, mode)
40     print("Decrypted message:", decrypted_message, '\n')
41

```

Mode AES.MODE_ECB
Encrypted message: EbbGhFgG0uP/4aWsx+wLGDt8NesDCbAgP0f00qXwEpX6TmEjPBX/tr5jRlie87bVCp4uTKlgrI6p9S/20QipA==
Decrypted message: I'm moving up and down, side to side like a roller coaster

Mode AES.MODE_CBC
Encrypted message: 8Rbf133t9TFE9judx2YMfPkwx4Mxo6cwYDeLQCYbdIHTTqNHN/i2bdtehhlBtglQkKqY5ZN5bC5VjPiN2091zS5a+IWy73YqzR
Decrypted message: I'm moving up and down, side to side like a roller coaster

Mode AES.MODE_CFB
Encrypted message: dXcbiqiv43NiN0hAhXTesh06JpRMnvmsmYydUqMIRtFl8htleii1UJH42ilgVxk9Ej9nfAJJ/GsJPQxQ38woeej72T+rq9UkZA=
Decrypted message: I'm moving up and down, side to side like a roller coaster

Mode AES.MODE_OFB
Encrypted message: SZ13D0f1Yx4a+1ZP0RBEEnmlNoKoLB311pHdB3PUGzTolIUP8TGEu7oqjlde/1dzCDNNYvwSkdCDF743rC8lgWtHiLhWnm7hPLY8=
Decrypted message: I'm moving up and down, side to side like a roller coaster

Mode AES.MODE_CTR
Encrypted message: qYRi4mntG/RdtZHZCiiwo+da6y9rrF0JZPWtbitdh05aPyX9AtbmJ/7w7562z8Weg//7oV9XubTmWFsvHkD3ZX1JwhZu1Q==
Decrypted message: I'm moving up and down, side to side like a roller coaster

Mode AES.MODE_CCM
Encrypted message: OnQjQPMHQFo0pNYXTMz98ynU409MRbLkK7FgwQ/R1ZkAEnUfjHVRAMKdy1rFsJEPZ0y5spl4R0/DRssMsqz1Krhzee/4L7cgZ7J0q
Decrypted message: I'm moving up and down, side to side like a roller coaster

Mode AES.MODE_GCM
Encrypted message: k6jnHlewKzqHmowcsXKbml1nL9UdD4Re1LSr+QPsYkrCh8/fpekKBuU9ivDPf2437WAWlhrF6o6oya10/thgRYKchCEb+EmdsJvIi
Decrypted message: I'm moving up and down, side to side like a roller coaster

Mode AES.MODE_OCB
Encrypted message: TpCaArcVdEwpUWtQ9cuerjl/8nmxgqzb2cGWuBn58AWglpx8GUsYVXZlwY6Aczk5+aq2HlflFd/eivWMCPQEsZ0BEa8N6imXL36nF
Decrypted message: I'm moving up and down, side to side like a roller coaster

Rivest, Shamir, Adleman (RSA)

Available functions:

- `generate_rsa_keys(bits)` → `private_key, public_key, rsa_components`
- `save_rsa_keys(private_key, public_key, private_key_filename, public_key_filename)` → `None`
- `import_rsa_key(key_filename)` → `RSA key object`
- `read_rsa_key(key_string)` → `RSA key object`
- `encrypt_rsa(message, public_key)` → `encrypted_message`
- `decrypt_rsa(encrypted_message, private_key)` → `message`
- `sign_message_pss(message, private_key)` → `signature`
- `verify_signature(message, signature, public_key)` → `True/False`

Supported RSA key sizes (in bits):

- 1024
- 2048
- 3072
- 4096

```

1 # RSA docs: https://pycryptodome.readthedocs.io/en/latest/src/public_key/rsa.html
2 # RSASSA-PSS docs: https://pycryptodome.readthedocs.io/en/latest/src/signature/pkcs1_pss.html
3
4 def generate_rsa_keys(bits=2048):
5     """Generates a pair of RSA keys (public and private).
6
7     Args:
8         bits: The number of bits for the RSA key.
9
10    Returns:
11        A tuple containing the private key and public key.
12    """
13    new_key = RSA.generate(bits)
14    private_key = new_key.export_key(format='PEM')
15    public_key = new_key.publickey().export_key(format='PEM')
16    return private_key, public_key, {'n': new_key.n, 'e': new_key.e, 'd': new_key.d, 'p': new_key.p, 'q':
17
18
19 def save_rsa_keys(private_key, public_key, private_key_filename="private_rsa.pem", public_key_filename=
20     """Saves the RSA keys to files.
21
22    Args:
23        private_key: The private key.
24        public_key: The public key.
25        private_key_filename: The filename for the private key file.
26        public_key_filename: The filename for the public key file.
27    """
28    with open(private_key_filename, 'wb') as f:
29        f.write(private_key)
30    with open(public_key_filename, 'wb') as f:
31        f.write(public_key)
32
33
34 def import_rsa_key(key_filename):
35     """Imports an RSA key from a file.
36
37    Args:
38        key_filename: The filename of the key file.
39
40    Returns:
41        The RSA key object.
42    """
43    with open(key_filename, 'rb') as f:
44        key = RSA.import_key(f.read())
45    return key
46
47 def read_rsa_key(rsa_key):
48     """Reads an RSA key from a string.
49
50    Args:
51        rsa_key: The RSA key as a string.
52
53    Returns:
54        The RSA key object.
55    """
56    return RSA.import_key(rsa_key)

```

```

57
58
59 def encrypt_rsa(message, public_key):
60     """Encrypts a message using RSA.
61
62     Args:
63         message: The message to encrypt.
64         public_key: The RSA public key.
65
66     Returns:
67         The encrypted message.
68     """
69     cipher_rsa = PKCS1_OAEP.new(public_key)
70     encrypted_message = cipher_rsa.encrypt(message.encode())
71     return binascii.hexlify(encrypted_message).decode('ascii')
72
73
74 def decrypt_rsa(encrypted_message, private_key):
75     """Decrypts a message using RSA.
76
77     Args:
78         encrypted_message: The encrypted message.
79         private_key: The RSA private key.
80
81     Returns:
82         The decrypted message.
83     """
84     cipher_rsa = PKCS1_OAEP.new(private_key)
85     decrypted_message = cipher_rsa.decrypt(binascii.unhexlify(encrypted_message))
86     return decrypted_message.decode()
87
88 def sign_message_pss(message, private_key):
89     """Signs a message using PKCS#1 PSS (RSASSA-PSS).
90
91     Args:
92         message: The message to sign.
93         private_key: The RSA private key.
94
95     Returns:
96         The signature.
97     """
98     hash_obj = SHA256.new(message.encode())
99     signer = pss.new(private_key)
100    signature = signer.sign(hash_obj)
101    return binascii.hexlify(signature).decode('ascii')
102
103
104 def verify_signature_pss(message, signature, public_key):
105     """Verifies an PKCS#1 PSS (RSASSA-PSS) signature.
106
107     Args:
108         message: The message that was signed.
109         signature: The signature to verify.
110         public_key: The RSA public key.
111
112     Returns:
113         True if the signature is valid, False otherwise.
114     """
115     hash_obj = SHA256.new(message.encode())
116     verifier = pss.new(public_key)
117     try:
118         signature = binascii.unhexlify(signature)
119         verifier.verify(hash_obj, signature)
120         return True
121     except ValueError:
122         return False

```

Example Usage

```

1 # List of possible RSA key lengths in bits
2 key_lengths = [1024, 2048, 3072, 4096]
3
4 # Index to select the desired key length (change kl_idx to use a different key length)

```

```

5 kl_idx = 0
6
7 # Message to be encrypted and signed
8 message = "Must be the water"
9
10 # Generate RSA keys (private and public) with the selected key length
11 private_key, public_key, rsa_components = generate_rsa_keys(bits=key_lengths[kl_idx])
12
13 # Save the generated RSA keys to files for later use
14 save_rsa_keys(private_key, public_key)
15
16 # Import (load) the saved public key from file
17 public_key_imported = import_rsa_key("public_rsa.pem")
18
19 # Import (load) the saved private key from file
20 private_key_imported = import_rsa_key("private_rsa.pem")
21
22 # Print the private and public keys as decoded strings
23 print(f"Private key: {private_key.decode()} \n")
24 print(f"Public key: {public_key.decode()} \n")
25
26 # Encrypt the message using the imported public key
27 encrypted_message = encrypt_rsa(message, public_key_imported)
28
29 # Decrypt the encrypted message using the imported private key
30 decrypted_message = decrypt_rsa(encrypted_message, private_key_imported)
31
32 # Display original, encrypted, and decrypted messages
33 print("Original message:", message)
34 print("Encrypted message:", encrypted_message)
35 print("Decrypted message:", decrypted_message, '\n')
36
37 # Generate a digital signature for the message using PSS (Probabilistic Signature Scheme)
38 signature = sign_message_pss(message, private_key_imported)
39 print(f"Signature: {signature}")
40
41 # Verify the signature using the public key
42 is_valid = verify_signature_pss(message, signature, public_key_imported)
43 print(f"Signature is valid: {is_valid}")
44

```

```

✂ Private key: -----BEGIN RSA PRIVATE KEY-----
MIICXAIBAAKBgQCnQsw4PqE5XyLovxdLR4yanDsY+xt4n46kVqIup/tsezaMevd/
pvJGXhly8h7arrx6YbH4V8QISzMhZSqZRxFXhCdx0gyRfJ1INxyYVn6bcyyoFP
1IpTnbvd500i6ygyBrPi35UPAtMuwwt8ghkD66Bv+ra8vE4WUPFS4pWb24QIDAQAB
AoGAAGSNncrfxhh4P8TpZ77WyzGPXobeML/DCubh/S47oXsSyi1XmGjPr0Sa61Gn
H1nB42NZvKE+MH9zs+Mg5dsC7Bn7rFqn/U3Lu8Tbexw0Ckc52FjpxrkZ0k5fesYs
x3Jdrw46Z0o5YNlqdw5VlovW7LrxpJWDmWbt6LHhINLIm0CQC7J92Xh2oBfauh
VtWUkaugrVpbgEnx0T1UP4tU6aYwG4D6MAaEMKEFbFvJNeHlljoQDMFht3XzxsYo
KUJJoI4adAkEAwcdagvRM77b6l0MpnX8kfxZGv7y18pcF6m8au6YcVSfzC1LUEH6f
uI7Au77uRAYm2cRQ44ZruezxRCDgvX7cFQJAHmzGq0GKTB+/7Q50yA4+VmqYC9P5
MWSQkceawP0S1TNDbmisRoUZSCZrbXRI+s3HNqC8N0R52l1wDutXzmMQSQJALaZY
FueWJFa33BCPajH8mA/YZ00A9ABN/Xd1FdXBEPKTWBSQLr6Vw/WlQ9Vr7g4I9H5C
DAersyY1hDY2m42aLQJBAIqYJ8AFFK2j02c0PhFKfppP+YwoPOTQ5XWeXXscoLuu
LZkP0EK7gGhJ+CF54V1JBPyvvnLx2Hhsw82zw7Lk/VA=
-----END RSA PRIVATE KEY-----

Public key: -----BEGIN PUBLIC KEY-----
MIGfMA0GCsGqSIb3DQEBAQUAA4GNADCBiQKBgQCnQsw4PqE5XyLovxdLR4yanDsY
+xt4n46kVqIup/tsezaMevd/pvJGXhly8h7arrx6YbH4V8QISzMhZSqZRxFXhC
Dx0gyRfJ1INxyYVn6bcyyoFP1IpTnbvd500i6ygyBrPi35UPAtMuwwt8ghkD66Bv+
ra8vE4WUPFS4pWb24QIDAQAB
-----END PUBLIC KEY-----

Original message: Must be the water
Encrypted message: 31f4a50e792d8e2d3bf5307181bacc8f32f8722fd93e9e8c436e726f600723c8e19f6ff37d39848a26f8ccc24cc3608eb2649
Decrypted message: Must be the water

Signature: 650125ac52cd038a5ec1dd652711cc878beb53c2a2b79a447bc8291cf1eb391babfe979e873df1852f8f74f55f1af8f3f71972b9bc566
Signature is valid: True

```

✧ Elliptic Curves Digital Signature Algorithm (ECDSA)

Available functions:

- `generate_ecc_keys(curve) → private_key, public_key`
- `import_ecc_key(key_filename) → ECC key object`

- `read_ecc_key(key_string)` → ECC key object
- `export_ecc_key(key)` → None
- `sign_message_ecc(message, private_key)` → signature
- `verify_signature_ecc(message, signature, public_key)` → True/False

Supported curves:

- NIST P-192
- NIST P-224
- NIST P-256
- NIST P-384
- NIST P-521
- Ed25519
- Ed448

```

1 # ECC docs: https://pycryptodome.readthedocs.io/en/latest/src/public_key/ecc.html
2 # ECDSA docs: https://pycryptodome.readthedocs.io/en/latest/src/signature/dsa.html
3
4 def generate_ecc_keys(curve='NIST P-192'):
5     """Generates a pair of ECC keys (private and public).
6
7     Returns:
8         A tuple containing the private key and public key.
9     """
10    key = ECC.generate(curve=curve)
11    private_key = key.export_key(format='PEM')
12    public_key = key.public_key().export_key(format='PEM')
13    return private_key, public_key
14
15
16 def import_ecc_key(key_filename):
17     """Imports an ECC key from a file.
18
19     Args:
20         key_filename: The filename of the key file.
21
22     Returns:
23         The ECC key object.
24     """
25    with open(key_filename, 'rb') as f:
26        key = ECC.import_key(f.read())
27    return key
28
29
30 def read_ecc_key(key):
31     """Reads an ECC key from a string.
32
33     Args:
34         key: The ECC key as a string.
35
36     Returns:
37         The ECC key object.
38     """
39    return ECC.import_key(key)
40
41 def export_ecc_key(key):
42     """Exports an ECC key to PEM format.
43
44     Args:
45         key: The ECC key object.
46
47     Returns:
48         The ECC key in PEM format.
49     """
50    return key.export_key(format='PEM')
51
52
53 def sign_message_ecc(message, private_key):
54     """Signs a message using ECDSA.
55
56     Args:
57         message: The message to sign.
58         private_key: The ECC private key.

```

```

59
60 Returns:
61     The signature.
62 """
63 hash_obj = SHA256.new(message.encode())
64 signer = DSS.new(private_key, 'fips-186-3')
65 signature = signer.sign(hash_obj)
66 return signature
67
68 def verify_signature_ecc(message, signature, public_key):
69     """Verifies an ECDSA signature.
70
71     Args:
72         message: The message that was signed.
73         signature: The signature to verify.
74         public_key: The ECC public key.
75
76     Returns:
77         True if the signature is valid, False otherwise.
78     """
79     hash_obj = SHA256.new(message.encode())
80     verifier = DSS.new(public_key, 'fips-186-3')
81     try:
82         verifier.verify(hash_obj, signature)
83         return True
84     except ValueError:
85         return False
86
87

```

▼ Example Usage

```

1 # List of available elliptic curves for ECC key generation
2 available_curves = ['NIST P-192', 'NIST P-224', 'NIST P-256',
3                     'NIST P-384', 'NIST P-521', 'Ed25519', 'Ed448']
4
5 # Index to select the desired elliptic curve
6 idx_curve = 0 # Change this index to use a different curve
7
8 # Generate an ECC key pair using the selected curve
9 private_key, public_key = generate_ecc_keys(available_curves[idx_curve])
10
11 # Print the generated private and public keys
12 print(f"Private key: {private_key}\n")
13 print(f"Public key: {public_key}\n")
14
15 # Message to be signed
16 message = "Valterri, it's James..."
17
18 # Sign the message using the private key (imported for use)
19 signature = sign_message_ecc(message, read_ecc_key(private_key))
20 print(f"Signature: {signature} \n")
21
22 # Verify the signature using the public key (imported for use)
23 is_valid = verify_signature_ecc(message, signature, read_ecc_key(public_key))
24 print(f"Signature is valid: {is_valid} \n")
25

```

```

🔑 Private key: -----BEGIN PRIVATE KEY-----
MG8CAQAwEwYHKoZIzj0CAQYIKoZIzj0DAQEeVBTAgEBBjKTWg+4P0TECAqteSH
N699vRgkHjuggYKhNAMyAASvMLRATWfvyYrmfXGkLhdnBu05efCskLZm9N7L4Hm2
9xTyUrUpZ0Mu4s0uHJ0wXYI=
-----END PRIVATE KEY-----

Public key: -----BEGIN PUBLIC KEY-----
MEkwEwYHKoZIzj0CAQYIKoZIzj0DAQEDMgAErzJUQE1n78mK5n1xpC4XZwbt0Xnw
rJJWZvTey+B5tvcU8lK1KWtjLuLDrhyTsF2C
-----END PUBLIC KEY-----

Signature: b'\x86\xc2KAXaeB\Xeekx\x1e\x16\x9dy\xb5\xb7\xa5h\x88\x96%\x95V\xc7\xe9\xe6\xb7\xdb\x10\xb2!L\x1b.\x04\x84\x8
Signature is valid: True

```

✓ Assignment

✓ Symmetric Encryption

In this assignment, students will explore AES encryption by performing the following tasks:

1. Encrypt a message: use AES with a chosen mode of operation and key length to encrypt a given message using a pre-shared secret key.
2. Publish to JSONbin: upload the encrypted message to JSONbin.
3. Retrieve and Decrypt: fetch an encrypted message previously published by your friend on JSONbin and decrypt it using the correct passphrase, key length, and mode of operation.
4. Analyze AES-GCM Behavior: encrypt and decrypt the same message multiple times using AES-GCM-128 and explain why the ciphertext varies.
5. Test Mode & Key Length Variations: encrypt a message using six different combinations of AES modes and key lengths, then analyze how these factors affect ciphertext length.
6. Discussion: briefly explain the impact of mode of operation selection, key length, IV/nonce, and authentication tags on ciphertext size and security.

```
1 # Pycryptodome AES docs: https://www.pycryptodome.org/src/cipher/aes
2
3 available_modes = {
4     1: AES.MODE_ECB,
5     2: AES.MODE_CBC,
6     3: AES.MODE_CFB,
7     5: AES.MODE_OFB,
8     6: AES.MODE_CTR,
9     8: AES.MODE_CCM,
10    11: AES.MODE_GCM,
11    12: AES.MODE_OCB,
12 }
13
14 available_AES_key_lengths = [128, 192, 256] # key length in bits
```

```
1 pre_shared_secret_password = "MySecretPassword123" # Add here the shared secret password you exchanged
2 message_to_be_exchanged = "Hello! This is my super secret message" # Add here the message you want to exchange
3
4 # T0-D0: Encrypt the message using AES, a mode of operation and key length of your choosing, and the pre-shared secret password
5 selected_mode = AES.MODE_GCM
6 selected_key_length = 256
7
8 my_ciphertext = encrypt(
9     message_to_be_exchanged,
10    pre_shared_secret_password,
11    selected_key_length,
12    selected_mode
13 )
14
15 print("Plaintext:", message_to_be_exchanged)
16 print("Ciphertext:", my_ciphertext)
17
18 my_payload = {
19     "ciphertext" : my_ciphertext,
20     "mode"       : "AES.MODE_GCM",
21     "key_length" : selected_key_length
22 }
23
24 # T0-D0: Publish the encrypted message in JSONbin
25 upload_resp = send_message_to_jsonbin(
26     JSONbin_Bin_ID_A,
27     my_payload,
28     JSONbin_X_Master_Key
29 )
30 print("Uploaded to my bin with ID:", upload_resp["metadata"]["parentId"])
31
32
33 # T0-D0: Retrieve the encrypted message from JSONbin (your friend message! Not the message you just published)
34 friend_payload = receive_message_from_jsonbin(
35     JSONbin_Bin_ID_B,
```

```

36     JSONbin_X_Master_Key
37 )
38
39 friend_ct = friend_payload["ciphertext"]
40 friend_mode = eval(friend_payload["mode"])
41 friend_klen = int(friend_payload["key_length"])
42
43 print("Retrieved friend's payload:", friend_payload)
44
45
46
47 # T0-D0: Decrypt the message from your friend
48 friend_plaintext = decrypt(
49     friend_ct,
50     pre_shared_secret_password,
51     key_length = friend_klen,
52     mode = friend_mode
53 )
54 print("Decrypted friend's message:", friend_plaintext)

```

 Plaintext: Hello! This is my super secret message
 Ciphertext: YLnGtP9D0py0/R30tBY3/zxtsvvAn/LLP0jdX4dNb7uu9Yzsk+M/erAJwC7+XZ+CCpGVZBSJMTcgei5ubcM4GLA
 Uploaded to my bin with ID: 6836102f8561e97a501c4973
 Retrieved friend's payload: {'ciphertext': 'lnyY01dUqwtY8yBWhDsgnY3c3KCITMA5Vrru5tX7Md9++iHe0VIY', 'mode': 'AES.MODE_GCM'}
 Decrypted friend's message: hello world

```

1 # T0-D0: Try to encrypt and decrypt the same message multiple times using AES-GCM-128. Why is the cipher
2 repeat_msg = "This text never changes"
3 repeat_runs = 3
4 repeat_cts = []
5
6 for _ in range(repeat_runs):
7     ct = encrypt(repeat_msg, pre_shared_secret_password,
8                 key_length = 128,
9                 mode = AES.MODE_GCM)
10    repeat_cts.append(ct)
11
12 print("AES-GCM-128 outputs:")
13 for i, c in enumerate(repeat_cts, 1):
14     print(f"Run {i}: {c}")
15
16 # AES-GCM prepends a new 12-byte nonce (IV) to every encryption call
17 # Because the nonce is randomly generated each time, the [nonce|cipher|tag]
18 # blob is always different even with identical plaintext + key + passphrase
19 # This randomness is needed, because reusing a nonce in GCM would break security
20
21 # T0-D0: Try different combinations of modes of operation and key lengths (6 combinations in total is su
22 test_combinations = [
23     (AES.MODE_ECB, 128),
24     (AES.MODE_CBC, 128),
25     (AES.MODE_CFB, 256),
26     (AES.MODE_OFB, 192),
27     (AES.MODE_CTR, 128),
28     (AES.MODE_CCM, 256),
29 ]
30
31 print("\nMode\tKeyLen\tCiphertext Length")
32 for mode, klen in test_combinations:
33     ct = encrypt("mode-length comparison message",
34                 pre_shared_secret_password,
35                 key_length = klen,
36                 mode = mode)
37     print(f"{mode}\t{klen}\t{len(ct)}")
38
39
40 # T0-D0: Briefly discuss how different combination of mode of operation and key length affect ciphertext
41
42 # The ECB mode (mode 1) results in the shortest ciphertext (44) because it does not use
43 # an IV or authentication tag. However, it is insecure because identical plaintext
44 # blocks result in identical ciphertext blocks.
45 #
46 # Modes like CBC (2), CFB (3), and OFB (5) include a 16-byte IV, which is
47 # prepended to the ciphertext. This increases the total size to around 64 characters.
48 #

```

```

49 # CTR mode uses a 12-byte nonce instead of a full 16-byte IV, so its ciphertext is slightly
50 # shorter than CBC-based modes.
51 #
52 # AEAD modes such as CCM (8) include both a 12-byte nonce and a 16-byte authentication
53 # tag, making their ciphertext the longest (80 characters). The added
54 # tag ensures both confidentiality and integrity.
55 #
56 # The AES key length (128, 192, or 256 bits) does not change the ciphertext length. It only
57 # affects security strength, with longer keys being more resistant to brute-force attacks.
58

```

↗ AES-GCM-128 outputs:

```

Run 1: 2Xi1C0RGHe7RNS029LofjDLSAxZZSnVIDtwSXx0WaGSmr5YBbJ0uIke7tGF6sbChpH13
Run 2: 1PnUeg3rLGZk81V1jvPrQlhDkcpPXjP/1rVio2Q702oh4r5+S+P3ouRfqFRX39nxHjwG
Run 3: IeC0ArKa8cK/J5FrKZMLqRjSRbUwf8QBUIqbcy0hfnzJeJ4fIk+XVoJYmpYfFwEcnMFa

```

Mode	KeyLen	Ciphertext Length
1	128	44
2	128	64
3	256	64
5	192	64
6	128	56
8	256	80

↘ Asymmetric Encryption

In this assignment, students will work with RSA encryption by performing the following tasks:

1. Generate RSA Key Pairs: create a new pair of RSA private/public keys with a selected key length and save them.
2. Print Keys: display the generated private and public keys in readable format.
3. Import RSA Keys: load previously saved RSA keys from files for encryption and decryption.
4. Encrypt a Message: request a friend's public key, use it to encrypt a confidential message, and publish the ciphertext to JSONbin.
5. Retrieve and Decrypt: fetch an encrypted message previously published by your friend on JSONbin and decrypt it using the correct private key.
6. Experiment with Key Lengths: generate RSA keys with different key lengths (1024, 2048, 3072, 4096 bits) and analyze the key components (n, e, d, p, q).
7. Discussion: briefly compare how key length impacts security, performance, and ciphertext size.

```

1 rsa_key_lengths = [1024, 2048, 3072, 4096] # Available key lengths in bit
2
3 # T0-D0: Generate and save a new pair of RSA private/public keys
4 key_length = 2048
5 private_key, public_key, rsa_components = generate_rsa_keys(bits=key_length)
6
7 save_rsa_keys(private_key, public_key, private_key_filename="private_rsa.pem", public_key_filename="publ
8
9
10 # T0-D0: Print the just generated private and public keys
11 print(f"Private key:\n{private_key.decode()}\n")
12 print(f"Public key:\n{public_key.decode()}\n")
13

```

↗ Private key:

```

-----BEGIN RSA PRIVATE KEY-----
MIIeowIBAAKCAQEA0eabMwvML1dqYkSZDaSeUmhxXsldibYh/zveFJ5i3/Q94gvh
v/QaM043giGB+MPPrZfjBeLbVivQgxmKraf9hZZuvR+knQPn6+tEdR6Y+lGvH5+Q
W00CUWLK+prksZ+aWzjPzYrLIo1ZVYw5hFdt/m+HncFs4Yayx74Wqm6bF6a18qQz
lsp007E/Q7kqUGb4/asQcPM0+NpV5k1nu56bA+DJXB/aiajg/8JBkB+RGc4o0ocl
1YY4ZXqB/sJYrpmENQSMIVRRyD61ibMtE98EaEdejm2tJIp6i9MfSf2xCXcNkKQW
+DB/DSgjW0FCehPN32sw0JXv0gpfrI84LKL6wIDAQABAoIBAGKMTeqjm9qQRtB
cyaKJwEW4z92+FErL3wHbiWtFUwDfrhuDrASWejDr0E0fVjACnX8zUxardJckGm0
6Lz0Lg0N1PE7dYx4pH9J5sq8Mm04TSniPyHUsz8o2dieN4hDoHK3gb0FnL3jSlHt
tt0xPsXKNFh3U3jxRmfSufTz2CI600CDTK2ZRSBsJEVIW7XQBS6DCd91N7kymtGg
Ssz7prpFzi9jZ1PQ0CxWnFF9ebmAC5gIjM0RPNVvHbxC3taalkazaHz+ZI3VVXhR
rtIDRVxFFdgmumG5AdqHNeRdZQWmZ31axSozpxkUzvZuSPOGmdt3rmIMMdg++l0h
aF09B1ECgYEA0/E79C7b/giraIyCP45cpxwiV0cH0NrKpsUM0Cp5zF24MC3VirGI
Kd44QoF9brYPQdvEh1MmzWXRwUR/2AhtLXnH61iW0ZNFVLi3cTzEavypCDYntDrm
wYM0FsEyFeZLhubnKR3ZHQuCRT1jC6JvBH9FbPMiCNsbaRU/Lx0/AdsCgYEA/Yi6
+FeVDVP7tBwpSGW3Y0gRTQzWHNjEGgF5MoG18UPGccJnsSGq0FPShAI+Fd5Lxaji
qz3v1Pjro7e0DyLrLIU0mEPHGKI9PFr5L5Z5iHn98fQvIkM3pCt5oN328qJ1/4J
MerkJStPjEmIjzg5ZvzaX/0YURKGGAP1buK0UTECgYAf0y8nxp22rAq0ojo4KMSr
9a9uvKfkgLZwIRLXeiVbgeU076K9cLzAg9kZ/tbkELR2pvzpiVMGN/G1VbSQ/Pu
yE7f/TIzjfcLHZYZgvAazKxAdtSGdkXBd5ZLbtv3dP/hB0BTHIcBDMiLdBGfDre3
fxp7hqWxjtJNPtAhdzPrDwKBgHEd6A2VRfNEXTDj2Ttr5n1eDr0LeBw2uoe/0Isw
RDT4wDLR02JUWxnHzE0Jkxm+rJ/r0fYFFFQ5GylYZBFsyvB7u/SwCXPYjUm2zSb

```

```
mMryhcSPaho0K4H8m1UVowx10TsdJI0Y9+F9iWhWBo+fzjyc4/GfwsAP+7QLUB3B
GCKBAoGBAM23QJBq9o9j0hoRUjmoVSKSoQsl1SoAju61iR7DV7ui3WiFVTTK+nL5
xSkrFPYIMAAGuTWBdkl1kFv8suqWe0czAy4nRwjo5Z9rn21KFeNv7iKTJrk6wVND
gf2MEE/GwJX2F1jPqKexhSyZzhLLTveZ0LnjXL1jIQ7G5fAPbs1k
-----END RSA PRIVATE KEY-----
```

```
Public key:
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA0eabMwvML1dqYkSZDaSe
UmhxXsldibYh/zveFJ5i3/Q94gvhv/QaM043giGB+MPPrZfjBeLbVivQgxmKraf9
hZZuvR+knQPn6+tEdR6Y+lGvH5+QW00CUWLK+prksZ+aWZjPzYrLIo1ZVYw5hFdt
/m+HncFs4Yayx74Wqm6bF6a18qQzLsp007E/Q7kqUGb4/asQcPM0+NpV5k1nu56b
A+DJXB/aiajg/8JBkB+RGc4o0ocl1YY4ZXqB/sJYrpmENQSMIVRyD61ibMtE98E
aEdejM2tIip6i9MfSf2xCXcNkKQW+DB/DSgjW0FCEhPN32sw0JXv0gpfrI84LKGL
6wIDAQAB
-----END PUBLIC KEY-----
```

```
1 # T0-D0: Import your private and public RSA keys
2 my_private_key = import_rsa_key("private_rsa.pem")
3 my_public_key = import_rsa_key("public_rsa.pem")
4
5 # T0-D0: You want to send a confidential message to your friend... Ask them to share their public key wi
6 friend_public_key_str = ""-----BEGIN PUBLIC KEY-----
7 MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAwbdNqvRDIbHm3KHkG3Rb
8 niFdGzLlqYPHwDf6YcjYf9kHfL0zpjzM67/Vc8Jz/yBTZowcF6dntgwfDys/eph
9 yJQcqQTot+oNFjP9X5NKWggZoL93Xe/McLJ+446kWK8jl2U5M1HkzIMFPfbLPDDa
10 htMZ57PPTUe0wk0iqcmPoVxpokhiDZoMndDJXVn20tSPN++x9ku/j3urc/DmbfVX
11 4rhPrL7UudLxj3KDMYJByt7Q2a0WkI0lJ8XSz82o/g+PGim+ayolnPzc+xp4EuB
12 lkny2jk1BVCrWzNv5C1SrqtM37RnDq8Uj4GAF214DBjZhrKym4MvEWz40BgPxr3
13 2wIDAQAB
14 -----END PUBLIC KEY-----""
15
16 friend_public_key = read_rsa_key(friend_public_key_str)
17
18 # T0-D0: Encrypt the message using your friend public key and publish the encrypted message in JSONbin
19 message_to_friend = "Hi, this is my RSA-encrypted message"
20 encrypted_for_friend = encrypt_rsa(message_to_friend, friend_public_key)
21 rsa_payload = {
22     "ciphertext": encrypted_for_friend,
23     "scheme": "RSA",
24     "key_length": my_public_key.size_in_bits()
25 }
26
27 upload_response = send_message_to_jsonbin(JSONbin_Bin_ID_A, rsa_payload, JSONbin_X_Master_Key)
28 print("Message encrypted and uploaded:", upload_response)
29 # T0-D0: Retrieve the encrypted message from JSONbin (your friend message! Not the message you just publ
30 received_rsa_payload = receive_message_from_jsonbin(JSONbin_Bin_ID_B, JSONbin_X_Master_Key)
31
32 # T0-D0: Decrypt the message from your friend using your own private key
33 received_cipher = received_rsa_payload["ciphertext"]
34 decrypted_message = decrypt_rsa(received_cipher, my_private_key)
35
36 # Display original, encrypted, and decrypted messages
37 print("Decrypted message from friend:", decrypted_message)
38
```

Message encrypted and uploaded: {'record': {'ciphertext': '2bad4bad39f6def0b5da0a1594af71ed85344c1334b84bdbb9e274516a2ee
Decrypted message from friend: Alice and Bob were invented by the creators of RSA in one of their publications

```
1 # T0-D0: Try different RSA key lengths and inspect the RSA parameters (i.e., n, e, d, p, q)
2 for bits in rsa_key_lengths:
3     key = RSA.generate(bits)
4     print(f"\nRSA Key Length: {bits} bits")
5     print(f"n (modulus) : {key.n} (bits: {key.n.bit_length()})")
6     print(f"e (public exponent) : {key.e}")
7     print(f"d (private exponent) : {key.d} (bits: {key.d.bit_length()})")
8     print(f"p (prime 1) : {key.p} (bits: {key.p.bit_length()})")
9     print(f"q (prime 2) : {key.q} (bits: {key.q.bit_length()})")
10
```

RSA Key Length: 1024 bits
n (modulus) : 13519789048298887216622597767811549072205667187205377219146311780996061004343579108007245511744738
e (public exponent) : 65537
d (private exponent) : 7496668050340747386240828887540121348849343971045883721222663552099860673794294030087325006760122
p (prime 1) : 10137007500733695890957498693061153032650772554152145155620806912796396169846773668086418813467818

```

q (prime 2)      : 13337061304650659979244913432004122413488042111790436051391404660510457308388583055939008624413223

RSA Key Length: 2048 bits
n (modulus)      : 20161973166443504736328650975112050830213796132973283828995240706006739565262471336102319219589214
e (public exponent) : 65537
d (private exponent) : 16693713026053161501782408371039721860173153447603241296377797959070637623678622807493459398288642
p (prime 1)       : 13858238309709827414853357169212475580528570730520966152953458698970301291611254967466501019693218
q (prime 2)       : 14548727418200725775282158282582693393239970500777665765117998375603769229512812905905673040975398

RSA Key Length: 3072 bits
n (modulus)      : 39557134553457238791005614366302603773441833438577191941400246182003339673469187065242308246845896
e (public exponent) : 65537
d (private exponent) : 42999378451688262988407158818002006390588464747611870453413393014723254319513326617450631544094200
p (prime 1)       : 17941395361735145397104595894011192197252089614140872674832894241551265064670394887897208141565928
q (prime 2)       : 22047969935395033531057248644702229292003784280089325916974234959501931214531538636346738814665890

RSA Key Length: 4096 bits
n (modulus)      : 83647832850456369808030523870028029495545119752525034386119760434658097671748454661102690920223142
e (public exponent) : 65537
d (private exponent) : 26053395609502123208362040808962069549686661839089632793790829758341981151480313880247168445443869
p (prime 1)       : 27481579967949673107649808146676873405481915815168708102027852284235605890505310371439767114057909
q (prime 2)       : 30437781578792214550930136940609380333230870226330223627980170125269806253684796063159450859291200

```

▼ Digital Signatures

In this assignment, students will explore digital signatures and message authentication using RSASSA-PSS and ECDSA. They will complete the following tasks:

1. Sign and Encrypt a Message: encrypt a message using RSA, generate a digital signature, and publish both the message and the signature to JSONbin (use the same JSONbin).
2. Retrieve and Verify: fetch an encrypted message and its signature from JSONbin, verify its authenticity using the sender's public key, and decrypt it.
3. Key Usage Discussion: explain which keys were used for encryption/decryption versus signature generation/validation.
4. Signature Experiments: generate and compare signatures using different key types (e.g., RSA 1024-bit and 4096-bit, ECDSA NIST P-192, P-224, and P-521).
5. Key & Signature Analysis: briefly discuss how key type and length affect signature size, security, and performance in RSA vs. ECC.

```

1 # Now you want to be sure that the message you received is from your friend
2
3 # List of available elliptic curves for ECC key generation
4 available_curves = ['NIST P-192', 'NIST P-224', 'NIST P-256',
5                     'NIST P-384', 'NIST P-521', 'Ed25519', 'Ed448']
6
7 rsa_key_lengths = [1024, 2048, 3072, 4096] # Available key length in bit
8
9
10 # T0-D0: Follow the same procedure as for asymmetric encryption, but this time publish the signature along
11 # You can either use PKCS#1 PSS (RSA) or ECDSA to sign the message.
12
13 # message
14 signed_message = "Alice and Bob use RSA"
15
16 # generate RSA key pair (2048 bits)
17 rsa_priv_key, rsa_pub_key, rsa_params = generate_rsa_keys(2048)
18 save_rsa_keys(rsa_priv_key, rsa_pub_key)
19
20 # import the keys
21 rsa_priv = import_rsa_key("private_rsa.pem")
22 rsa_pub = import_rsa_key("public_rsa.pem")
23
24 # sign the message using RSA-PSS
25 rsa_signature = sign_message_pss(signed_message, rsa_priv)
26
27 # encrypt the message
28 friend_pub_key_str = ""-----BEGIN PUBLIC KEY-----
29 MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAwbDNqvRDIbHm3KHkG3Rb
30 niFdGzLlqYPHWdFd6YcjYf9kHfL0zpjzM67/Vc8Jz/yBTZowcF6dntgwfDys/eph
31 yJQcqQTot+oNFjP9X5NKWggZoL93Xe/McLJ+446kK8jl2U5M1HkzIMFPfblPDDa
32 htMZ57PPTUe0wk0iqcmPoVxpokhiDZoMndDJXVn20tSPN++x9ku/j3urc/DmbfVX
33 4rhPrL7UudLxj3KDMYJByt7Q2a0WkI0LJ8XSzz82o/g+PGim+ayoLnPzc+xp4EuB
34 lkny2jk1BVCrWzNv5C1SrqtM37RnDq8Uj4GAF214DBjZHrKym4MvEWz40BgPxr3
35 2wIDAQAB
36 -----END PUBLIC KEY-----""

```

```

37
38 friend_pub_key = read_rsa_key(friend_public_key_str)
39 encrypted_msg = encrypt_rsa(signed_message, friend_pub_key)
40
41 # upload to JSONbin
42 json_payload = {
43     "ciphertext": encrypted_msg,
44     "signature": rsa_signature
45 }
46 send_message_to_jsonbin(JSONbin_Bin_ID_A, json_payload, JSONbin_X_Master_Key)
47
48
49
50 # T0-D0: Discuss which keys you used for message encryption/decryption and which ones for signature generation
51
52 # Message encryption/decryption:
53 # - Message was encrypted using the recipient's PUBLIC RSA key.
54 # - Message was decrypted using the recipient's PRIVATE RSA key.
55
56 # Signature generation/validation:
57 # - The message was signed my PRIVATE key (sender)
58 # - The signature was verified using my PUBLIC key
59
60

```

```

➦ {'record': {'ciphertext':
'700f39b685482acf27048007dae83b0a7528d7ab64e4b6045b8d2927ce4612308b61b9649d37b17d4dbadace5d9892020cbff246698eb103b8c2f82
'signature':
'90f8504b97783012b9bb802f180385bedc57227a4ddc94937f803365daf70adf905effac8372d3caf85ab924ce2bcd7ffa17fd5c37c45d9e301221b
'metadata': {'parentId': '6836102f8561e97a501c4973', 'private': False}}

```

```

1 # T0-D0: Try to generate signatures with different combination of (RSA, RSA_KEY_LENGTH) and (ECDSA, CURVE)
2
3 test_message = "Signed by multiple schemes"
4
5 signature_tests = []
6
7 # RSA key tests
8 for bits in [1024, 4096]:
9     priv, pub, _ = generate_rsa_keys(bits)
10    signature_tests.append((f"RSA-{bits}", priv, pub))
11
12 # ECC curve tests
13 for curve in ["NIST P-192", "NIST P-224", "NIST P-521"]:
14     priv, pub = generate_ecc_keys(curve)
15     signature_tests.append((f"ECC-{curve}", priv, pub))
16
17 # Now sign and verify using the utility functions
18 for name, priv_key_str, pub_key_str in signature_tests:
19     if name.startswith("RSA"):
20         priv_key = read_rsa_key(priv_key_str)
21         pub_key = read_rsa_key(pub_key_str)
22         sig = sign_message_pss(test_message, priv_key)
23         verified = verify_signature_pss(test_message, sig, pub_key)
24         print(f"{name}: Signature length = {len(sig)} chars, Verified? {verified}")
25     else:
26         priv_key = read_ecc_key(priv_key_str)
27         pub_key = read_ecc_key(pub_key_str)
28         sig = sign_message_ecc(test_message, priv_key)
29         verified = verify_signature_ecc(test_message, sig, pub_key)
30         print(f"{name}: Signature length = {len(sig)} bytes, Verified? {verified}")
31
32
33 # T0-D0: Briefly discuss how key and signature sizes change between RSA and EC.
34
35 # RSA signatures are significantly larger than EC (Elliptic Curve) signatures.
36 # For example, a 1024-bit RSA key produces a 256-character signature,
37 # while an ECDSA key using NIST P-192 produces only a 48-byte signature.
38
39 # This size difference grows with RSA key length: 4096-bit RSA yields a 1024-char signature,
40 # while even the largest tested EC curve (P-521) yields only a 132-byte signature.
41
42 # Therefore, EC offers much smaller signature sizes for equivalent security levels.

```

43 # This results in faster signing/verification operations.
44

```
➦ RSA-1024: Signature length = 256 chars, Verified? True
RSA-4096: Signature length = 1024 chars, Verified? True
ECC-NIST P-192: Signature length = 48 bytes, Verified? True
ECC-NIST P-224: Signature length = 56 bytes, Verified? True
ECC-NIST P-256: Signature length = 64 bytes, Verified? True
```

Final discussion and report

The report should be divided in three sections addressing the following topics:

- **Symmetric encryption**
 - Describe the various steps you followed to encrypt, upload, download, and decrypt the secret messages.
 - Discuss which information you needed to share with your friend to enable the exchange of the encrypted messages.
 - Briefly discuss how different modes of operation and key lengths affect ciphertext length and security.
- **Asymmetric encryption**
 - Describe the various steps you followed to encrypt, upload, download, and decrypt the secret messages.
 - Discuss which information you needed to share with your friend to enable the exchange of the encrypted messages.
 - Regarding the previous point, compare with what was required when using symmetric encryption. What are the pros and cons of each cryptography scheme?
 - How does the RSA key length relate to the length (in bits) of the RSA components (i.e., n , p , q , e , d)?
- **Digital signatures**
 - Describe the various steps you followed to encrypt, sign, upload, download, decrypt, and verify the secret signed messages. In particular, highlight which private and public key were used for each operation.
 - Briefly discuss and compare (qualitatively) how PKCS#1 PSS and ECDSA differ in terms of: private and public key length, signature size, security, and performances.